



**UNIVERSITÀ DEGLI STUDI  
DI GENOVA**

# **NFT Marketplace: The Frontend Report**

**Course: DECENTRALIZED SYSTEMS**

**Prepared by :**

Musse Gebreezgabher : 6015033

**Submitted to :**

Professor Marina Ribaudo

Professor Matteo Dell'Amico

**February 18, 2025**

# Frontend Report: NFT Marketplace

## 1. Introduction

This report details the **frontend implementation** of the NFT Marketplace, including the **framework used, file structure, integration with the smart contract ABI, and functionalities** of various components. The frontend is responsible for **interacting with the blockchain**, displaying NFT details, allowing users to **mint, list, delist, and buy NFTs**, and ensuring a smooth user experience.

## 2. Framework Used

The frontend of the NFT Marketplace is built using **Vue.js**, a modern JavaScript framework known for its:

- **Component-based structure** (efficient UI updates)
- **Reactive data binding** (real-time updates)
- **Easy integration with Web3 libraries**

The application uses **Vue 3** for fast development and performance optimization.

### 2.1 Key Libraries and Dependencies

The following libraries were used for blockchain interactions and UI enhancements:

- **ethers.js** → Communicates with the Ethereum blockchain.
- **axios** → Fetches NFT metadata from IPFS.
- **Vue Router** → Manages different views in the application.

## 3. Project File Structure

Below is the **organized file structure** of the project:

```
NFT-Marketplace-Frontend/
├── src/
│   ├── assets/           # Static files (images, logos, etc.)
│   ├── components/       # Reusable Vue components
│   │   ├── Marketplace.vue # Lists NFTs for sale
│   │   └── Mint.vue       # NFT minting form
│   ├── router/           # Vue Router configuration
│   ├── web3.js           # Blockchain connection logic
│   ├── App.vue           # Root component
│   └── main.js           # Entry point
```

```
| — public/           # Static files
| — package.json      # Dependencies and scripts
| — vite.config.js    # Vite configuration
```

## 4. Smart Contract ABI Integration

### 4.1 What is the ABI?

The **ABI (Application Binary Interface)** defines the methods that can be called on the smart contract, allowing the frontend to interact with it. It includes:

- **Function names** (e.g., `mintNFT`, `listNFT`, `buyNFT`)
- **Inputs & outputs** (e.g., token ID, price, URI)
- **Event signatures** (e.g., `Minted`, `Listed`, `Purchased`)

### 4.2 ABI File (`NFTMarketplace.json`)

The ABI file is generated from the compiled smart contract and stored in:

```
src/
| — abi/
|   | — NFTMarketplace.json # Smart contract ABI
```

The frontend imports it as:

```
import contractABI from "../abi/NFTMarketplace.json";
```

This allows **minting, listing, and purchasing** NFTs directly from the UI.

## 5. Key Functionalities

### 5.1 Wallet Connection (Web3.js & MetaMask)

The `web3.js` component handles the **MetaMask** connection:

```
export async function connectWallet() {
  try {
    if (typeof window.ethereum === "undefined") {
```

```

    alert("MetaMask not detected. Please install MetaMask.");
    throw new Error("No crypto wallet found");
  }

  // Request account access
  await window.ethereum.request({ method: "eth_requestAccounts" });

  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer = await provider.getSigner();
  const address = await signer.getAddress();

  console.log("Connected wallet:", address);
  return { provider, signer, address };
} catch (error) {
  console.error("Wallet connection error:", error);
  alert(
    "Failed to connect wallet. Make sure MetaMask is installed and you are logged in."
  );
}
}

```

✓ Allows users to **connect their wallet** to the marketplace.

## 5.2 Minting an NFT (**Mint.vue**)

```

async mintNFT() {
  try {
    this.mintStatus = "Uploading NFT metadata to IPFS...";
    const tokenURI = await this.uploadToIPFS();
    if (!tokenURI) return;

    const { signer } = await connectWallet();
    const contract = await getContract(signer);

    const priceInWei = ethers.parseEther(String(this.nft.price)); //
    Convert price to string before parsing
    const txn = await contract.mintNFT(

```

```

        tokenURI,
        priceInWei,
        this.nft.rarity,
        this.nft.traits
    );
    await txn.wait();

    this.mintStatus = " NFT Minted Successfully!";
} catch (error) {
    console.error("Minting Error:", error);
    this.mintStatus = "❌ Error minting NFT.";
}
},

```

✅ Uploads metadata to IPFS and mints an NFT.

### 5.3 Listing an NFT (Marketplace.vue)

```

async listNFT(tokenId, price) {
    try {
        if (!price || isNaN(price)) {
            alert("⚠️ Price must be a valid number.");
            return;
        }

        const { signer } = await connectWallet();
        const contract = await getContract(signer);
        const txn = await contract.listNFT(
            tokenId,
            ethers.parseEther(price.toString())
        ); // Convert ETH to Wei
        await txn.wait();

        alert("NFT Listed Successfully!");
        await this.loadNFTs(); // Refresh Marketplace
    } catch (error) {
        alert("⚠️ Error listing NFT.");
    }
}

```

✓ Allows owners to list their NFT for sale.

### 5.3 Listing an NFT (Marketplace.vue)

```
async delistNFT(tokenId) {
  try {
    const { signer } = await connectWallet();
    const userAddress = await signer.getAddress();
    const contract = await getContract(signer);

    // Check ownership before delisting
    const nftOwner = await contract.ownerOf(tokenId);
    if (nftOwner.toLowerCase() !== userAddress.toLowerCase()) {
      alert("You can only delist NFTs you own.");
      return;
    }

    const txn = await contract.delistNFT(tokenId);
    await txn.wait();

    alert("NFT Delisted Successfully!");
    await this.loadNFTs(); // Refresh marketplace
  } catch (error) {
    console.error(`Failed to delist NFT ${tokenId}:`, error);
    alert("⚠ Error delisting NFT.");
  }
},
```

✓ Allows owners to delist their NFT from the selling list.

### 5.4 Buying an NFT (Marketplace.vue)

```
async buyNFT(tokenId, price) {
  const { signer } = await connectWallet();
  const contract = await getContract(signer);
  const txn = await contract.buyNFT(tokenId, {
    value: ethers.parseEther(price),
  });
  await txn.wait();
}
```

```
    alert("NFT Purchased!");  
    await this.loadNFTs();  
  },
```

✓ Transfers ownership and sends ETH to the seller.

## 6. Conclusion

The **NFT Marketplace frontend**, built with **Vue.js**, integrates seamlessly with the smart contract using **ethers.js** and **IPFS for metadata storage**. It enables users to **mint, list, buy, and manage NFTs** in an intuitive interface. The **smart contract ABI is used to interact with blockchain functions**, ensuring a decentralized and transparent NFT trading experience.